MODULE 3 · AGENTIC RAG · TECHNICAL CASE STUDY

# Semantic Search RAG vs. *Moment RAG*

Two ways to answer questions about a YouTube video from its transcript — a simple semantic-search baseline, and a moment-aware system that decomposes, retrieves hybrid, re-ranks, and cites the exact second.

CALVIN NGUYEN

GITHUB.COM/CALVNGUYEN/SEMANTIC_SEARCH_MOMENT_RAG

00 · THE PROBLEM

# An LLM can’t answer about a video it has *never seen*.

RAG (Retrieval-Augmented Generation) fixes that: find the relevant pieces first, then let the model answer using them.

# RAG in one slide: *retrieve*, then *generate*.

## The idea

Turn every passage into an *embedding* — a list of numbers capturing its meaning. Similar meanings sit close together in that number space.

To answer a question, embed the question too, find the *closest* passages, and hand them to the LLM as context. The model answers from those, not from memory.

## Why it beats keyword search

Keyword search needs the *exact words*. “Can I wear jeans?” would miss a passage titled “dress code.”

Semantic (embedding) search matches on *meaning*, so the question and the answer connect even with zero shared words. That is the core move in every slide that follows.

01 · THE SOURCE

# The video chosen — and *why* it fits.

A short, dense explainer is a clean knowledge source: many distinct ideas packed into a few minutes, each a natural “moment.”

WHY THIS VIDEO

# “How does a Vector Database work?”

It captures the exact shift this project is about: how vector databases transform traditional keyword-based search into **semantic understanding that actually comprehends meaning**, not just matching words.

- Its arc mirrors baseline → Moment RAG: SQL keyword lookup vs. meaning-based retrieval
- Topic-dense — embeddings, similarity, indexing, thresholds, chunk overlap — each a distinct moment
- Clean auto-captions → 312 timestamped cues; the system explains the very tech it runs on


youtube.com/watch?v=VVNYQKDLY5s

02 · THE REFERENCE

# Understanding the *Moment RAG* codebase.

The course reference puts intelligence on the query side and resolves retrieval to timestamped moments, not just documents.

# Enrich *once* at ingestion; think *hard* at query time.

### Ingestion (done once, up front)

Transcripts are stored as timed cues; every chunk keeps its *start/end timestamp* — that is what later makes it a citeable moment.

An LLM pre-tags each chunk with the *questions it answers*, and those are embedded too — a second way to find the chunk later.

### Query time (on every question)

The query is *decomposed* into sub-questions, then retrieved three ways — meaning, keywords, and stored questions — and the rankings are *fused*.

A re-ranker sharpens the shortlist, hits roll up to *moments*, and each citation deep-links into the video at the exact second.

# Four terms that do the heavy lifting.

## Finding candidates

*Dense / embeddings* — match on meaning (paraphrase-friendly).

*BM25 (sparse)* — classic keyword scoring; nails exact terms like “cosine” or “HNSW.”

*HyDE* — index the *questions a passage answers*, so a real question can match a stored question.

## Choosing the best

*RRF (Reciprocal Rank Fusion)* — merges several ranked lists into one, so meaning and keywords both get a vote.

*Cross-encoder re-rank* — reads the question and a passage *together* and scores the true match; slower but far more precise than embeddings alone.

03 · BASELINE

# Baseline: *flat* semantic search.

Fixed chunks, one embedding lookup, stuff-and-answer — the conventional RAG most teams ship first.

# Chunk → embed → index → *answer*

## 01 → 02 → 03 → 04

### Chunk

Cut the transcript into fixed ~256-token windows (25% overlap) → 13 chunks.

### Embed

Turn each chunk into a vector with text-embedding-3-small.

### Index

Store the vectors — NumPy cosine, or a real DB (ChromaDB / HNSW).

### Answer

Take the top-k closest chunks, hand them to gpt-4o-mini.

# Simple and fast — but it trusts *one* lookup.

### Where it struggles

Chunk boundaries fall wherever the token budget runs out — often *mid-thought*, splitting a explanation in half.

A single similarity lookup can grab chunks that are *about* the topic but don't actually contain the answer.

### And it can't cite

Multi-part questions only get one search, so one half tends to win and the other is under-answered.

Chunks have no clean topic-aligned timestamp, so answers can't point you to the moment in the video.

04 · MOMENT RAG

# Moment RAG: think on the *query* side.

Semantic moments with timestamps, question-indexing, hybrid retrieval, re-ranking, and cited answers.

# Intelligence moves to the *query* side

**01**

## Segment & Enrich

Split at topic shifts → 25 timestamped moments; tag each with the questions it answers (99 Q-vectors).

→

**02**

## Decompose

Break one question into 2–4 focused sub-queries.

→

**03**

## Hybrid + Rerank

Dense + questions + BM25, RRF-fused, then cross-encoder re-rank.

→

**04**

## Cite

Roll up to a gpt-4o-mini answer with &t= timestamp deep-links.

# Every stage removes a baseline failure mode.

### Better units, better recall

*Moments* break at topic shifts, so a cited span is a whole self-contained idea — and carries a real timestamp.

*Decompose* means a two-part question gets two searches, covering both halves instead of one.

### Better matching, better precision

*Hybrid + HyDE + RRF* blends meaning, exact keywords, and stored questions — so the right moment surfaces even with different wording.

The *cross-encoder* then re-reads each candidate against the question and promotes the true match before the answer is written.

BASELINE SEMANTIC SEARCH

## One lookup over arbitrary chunks

Fast and simple, but only as good as a single similarity match.

- 13 fixed chunks, boundaries set by token budget
- Single embedding lookup per query
- No timestamps to cite; can miss the “how”

MOMENT RAG

## Decompose, fuse, re-rank, cite

More moving parts, but retrieval lands on the right moment.

- 25 semantic moments aligned to topic shifts
- Sub-queries + dense + BM25 + questions, RRF + rerank
- Every citation deep-links to the exact second

BASELINE — FAILED

## “How does a vector database find similar vectors?”

“The context does not provide a specific answer to how a vector database finds similar vectors.”

— Retrieved chunks about vector DBs — but not the actual mechanism

MOMENT RAG — ANSWERED

## Cosine similarity, thresholds, chunk overlap

“…uses techniques such as cosine similarity for semantic search… scoring thresholds decide how similar a match must be… [1][3]”

— 6 citations, each a deep-link: 4:59, 8:10, 4:08, 1:04 …

“

The baseline said “the context does not provide an answer.” Moment RAG answered the same question — with six timestamped citations.

THE DIFFERENCE IS RETRIEVAL — NOT THE MODEL
BOTH USED GPT-4O-MINI ON THE SAME TRANSCRIPT

# One short video, measured both ways

**25**

semantic moments vs. 13 fixed chunks from the same transcript

`segment_moments()`

**99**

hypothetical-question vectors added as a HyDE retrieval branch

`enrich_moment()`

**~2¢**

total OpenAI cost for a full top-to-bottom run

`embeddings + gpt-4o-mini`

KEY FINDINGS

# Why moment-aware retrieval wins

Same source, same models — the gains come entirely from how retrieval is structured.

- Decomposition covers multi-part questions the baseline answers only halfway
- Question-indexing (HyDE) matches intent even when wording differs from the transcript
- Hybrid + RRF balances meaning (dense) against exact terms (BM25)
- Cross-encoder re-rank pushes the truly on-point moment to the top
- Timestamped citations make every claim verifiable in one click

# What it *doesn't* do yet — and where it goes next.

## Limitations

Higher cost & latency: one LLM enrichment call per moment, plus decomposition and re-rank on every query.

The NumPy index is exact but brute-force — fine for one video, not for millions of vectors.

Segmentation and answers inherit caption quality and the short explainer's depth.

## Improvements

Cache enrichment and persist vectors (ChromaDB / Qdrant) so ingestion runs only once.

Add self-query metadata filters and scale to a multi-video corpus.

Tune segmentation, top-k, and rerank depth; add an LLM-judge eval to measure quality.

MODULE 3 · AGENTIC RAG

# Better retrieval, *not* a bigger model.

Calvin Nguyen ·
github.com/calvnguyen/semantic_search_moment_rag